

HVAC waste detection based on rapid temperature changes

Internet of Things Project - Year 2023-2024

Leonardo Dessì
Università di Bologna
Bologna, Italy

leonardo.dessi@studio.unibo.it
Student ID number: 0001141270

Abstract—This paper presents a comprehensive Internet of Things (IoT) solution for monitoring and controlling HVAC systems with intelligent energy waste detection capabilities. The system integrates Arduino-based sensors, MQTT communication protocols, machine learning algorithms, and real-time data visualization to provide automated detection of energy wastage through rapid temperature changes. The architecture comprises four main components: firmware for IoT devices, a data proxy for communication bridging, a data analytics engine with predictive capabilities, and visualization dashboards. The system employs ARIMA-based temperature prediction models to forecast indoor temperature patterns and automatically detects potential energy waste scenarios when HVAC systems are active. Performance evaluation demonstrates the system's effectiveness in real-time monitoring, prediction accuracy across multiple time horizons, and reliable alert generation for energy optimization.

Index Terms—Internet of Things, HVAC monitoring, energy efficiency, machine learning, ARIMA prediction, MQTT communication, InfluxDB, temperature forecasting, energy waste detection

I. INTRODUCTION

Energy efficiency in building management systems has become increasingly critical due to rising energy costs and environmental concerns. Heating, Ventilation, and Air Conditioning (HVAC) systems typically account for 38% of total building energy consumption, making them prime targets for optimization strategies [1]. Traditional HVAC control systems often operate on simple thermostatic controls without considering external environmental factors, occupancy patterns, or predictive analytics, leading to significant energy waste.

The rapid advancement of Internet of Things (IoT) technologies presents unprecedented opportunities for intelligent building management systems. IoT-enabled sensors can provide continuous monitoring of environmental conditions, while machine learning algorithms can analyze patterns and predict future states to optimize system operation. The integration of real-time data analytics with predictive modeling enables proactive energy management rather than reactive control.

This project addresses the challenge of HVAC energy waste detection through the development of a comprehensive IoT monitoring system. The system focuses specifically on detecting energy waste scenarios based on rapid temperature changes

that indicate inefficient HVAC operation. By combining real-time sensor data with predictive analytics, the system can identify situations where HVAC systems are operating unnecessarily or inefficiently.

The primary objectives of this work are: (1) to develop a scalable IoT architecture for continuous HVAC monitoring, (2) to implement time-series forecasting models for temperature prediction and anomaly detection, (3) to create an integrated communication framework using MQTT protocols for real-time device control, and (4) to provide comprehensive visualization and alerting capabilities for system operators.

The system's innovation resides in its comprehensive approach to energy waste detection, which integrates diverse data sources such as indoor and outdoor temperature sensors, external weather APIs, and HVAC system status information to facilitate intelligent inference of energy consumption patterns. The use of ARIMA-based forecasting models enables the system to predict temperature trends and identify deviations that indicate potential waste scenarios.

II. PROJECT'S ARCHITECTURE

The IoT HVAC monitoring system is designed as a comprehensive, end-to-end solution that emphasizes modularity and scalability. It provides an integrated infrastructure for environmental sensing and control, effectively linking physical devices with advanced data analytics and user-facing interfaces.

A. System Overview

The architecture adopts a **hub-and-spoke model**, in which IoT devices collect environmental data and transmit it to a centralized proxy. The system supports **bidirectional communication**: data is sent from sensors to the proxy, and control commands are issued in the reverse direction. Notably, data from the ESP32 devices is transmitted via **HTTP**, while control commands from the proxy to the ESP32 are delivered using the **MQTT protocol**.

The data flow begins at the sensor level, progresses through a communication layer managed by the data proxy, and is then forwarded to storage and analytics layers. Finally, data is rendered through user-facing presentation layers. Figure 1 illustrates these components and their interconnections.

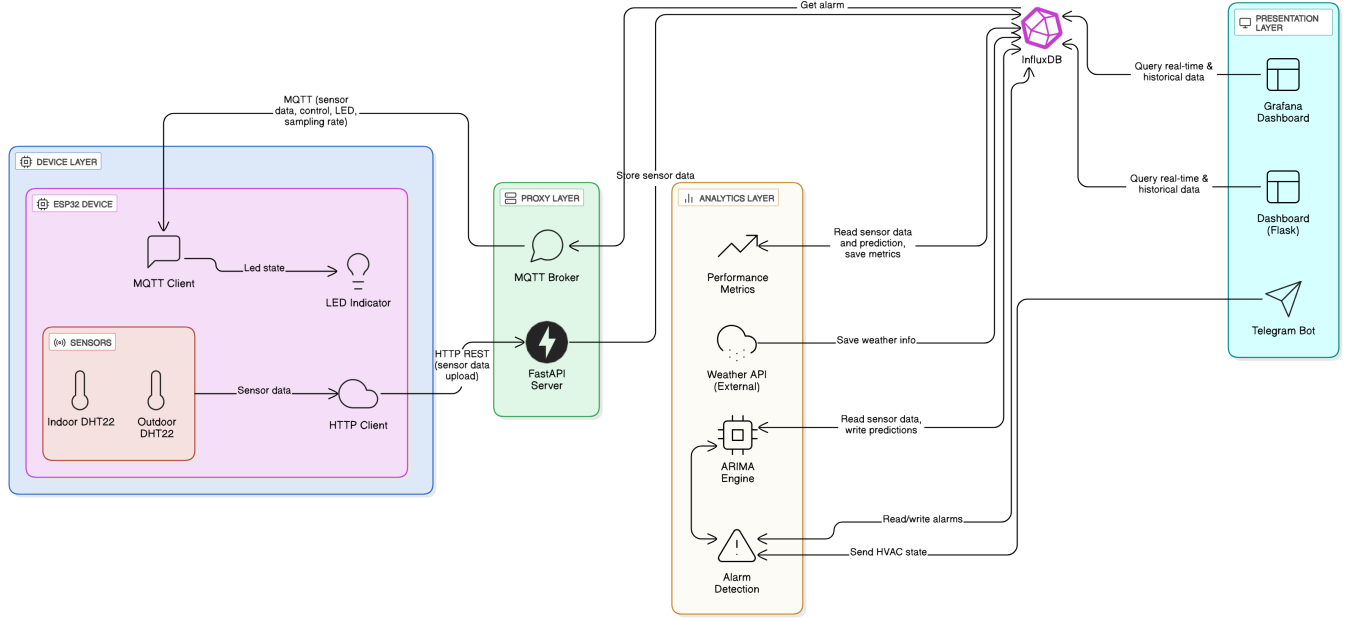


Fig. 1: System Architecture

B. Hardware Components

The system's hardware foundation consists of **ESP32 microcontrollers**, which function as the edge computing units. Each ESP32 interfaces with two **DHT22 sensors** to measure temperature and humidity in both indoor and outdoor settings, facilitating context-aware decision-making. An **LED indicator** connected to the ESP32 provides real-time visual feedback regarding system status.

C. Software Components

The software architecture follows a **multi-tiered design** to separate concerns effectively. The base layer comprises **Arduino-compatible firmware** running on the ESP32, which manages sensor readings and network communication. A **data proxy layer**, implemented in Python with FastAPI, bridges edge devices and higher-order services. This proxy conducts initial data validation, exposes a GUI interface, and forwards control commands.

An **analytics backend** processes time-series data using pipelines that incorporate prediction models and system diagnostics. This backend leverages **InfluxDB** for efficient storage of temporal data. Visualization is achieved through web technologies, while a **Telegram bot** provides a mobile access point. The system also integrates **weather APIs** and cloud services via **InfluxDB Cloud (EU Central)**.

D. Network Topology

The network employs a **hybrid topology** for connectivity. The ESP32 devices use onboard **Wi-Fi** to connect to a local network. **HTTP** is employed for transmitting sensor data from the ESP32 to the proxy, using POST requests with JSON payloads. In contrast, **MQTT** is used exclusively for sending control commands from the proxy back to the ESP32 devices.

E. Data Flow

Environmental data originates at the **DHT22 sensors**, which record measurements at user-defined intervals (default: 5 seconds). The ESP32 formats these readings into **JSON** and transmits them to the proxy via **HTTP POST** requests, including timestamps for latency analysis. The proxy validates and forwards the data to **InfluxDB** for storage.

The **Analytics Backend** retrieves stored data, performs aggregations and transformations, and generates short-term temperature forecasts (1, 10, 20, and 30 minutes). Forecast accuracy is continuously assessed against real readings. If anomalies indicative of energy inefficiency are detected, commands are issued through **MQTT topics** to trigger system responses, completing the sense-analyze-act cycle.

F. Block Architecture

From a block-level perspective, the system exhibits clear functional stratification:

- **Edge Layer:** ESP32 and sensors.
- **Communication Layer:** HTTP (sensor data to proxy), MQTT (commands to devices).
- **Storage Layer:** InfluxDB for time-series data.
- **Processing Layer:** Analytics and forecasting.
- **Presentation Layer:** Dashboards, GUIs, Telegram bot.
- **Integration Layer:** External APIs and cloud services.

III. PROJECT IMPLEMENTATION

The implementation of the IoT-based HVAC monitoring system integrates embedded hardware with a multi-layered software stack, resulting in a robust, scalable, and feature-rich architecture for environmental monitoring and energy efficiency analysis.

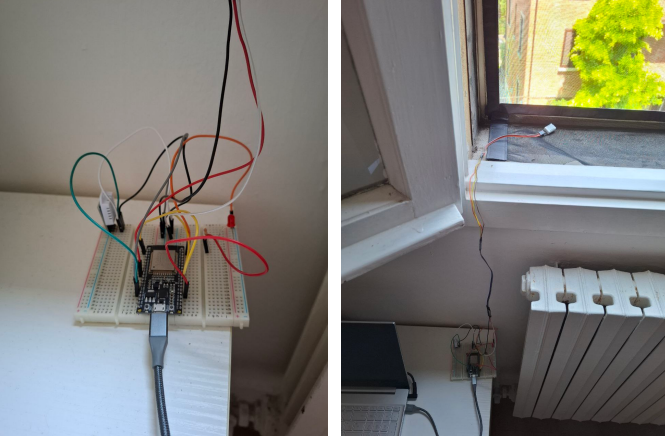


Fig. 2: ESP32 configuration

A. Hardware Configuration

At the core of the hardware setup lies the **ESP32 microcontroller**. Two **DHT22 sensors** are interfaced with the ESP32 to provide temperature and humidity measurements. The indoor sensor is connected to pin 13, while the outdoor sensor is connected to pin 25. Additionally, an **LED indicator** is connected to pin 2 to offer immediate visual feedback regarding the alarm state (if the led is on a waste is detected), thereby facilitating rapid diagnostics in the absence of a graphical interface.

The ESP32 establishes a connection to the local Wi-Fi network using credentials embedded within its firmware and maintains concurrent connections with both the HTTP server and the MQTT broker. To support high-precision time-series analytics, the system implements **NTP-based time synchronization** during the boot sequence and at predefined intervals.

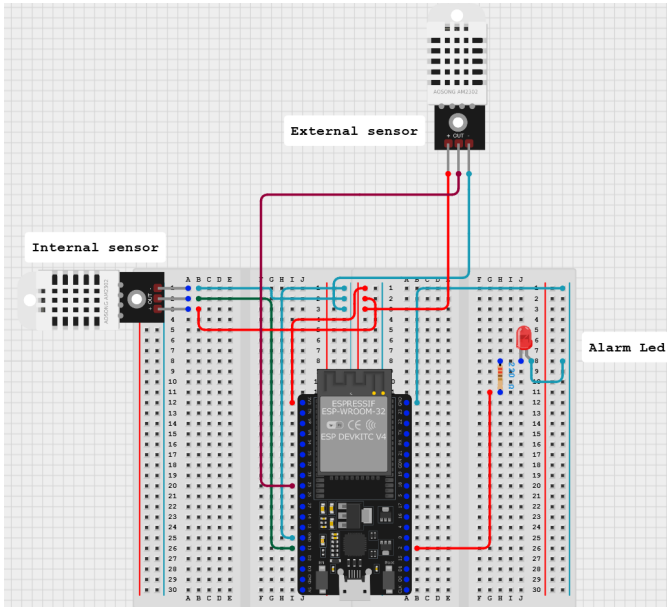


Fig. 3: Circuit scheme

B. Software Architecture

The software implementation spans several programming languages and frameworks, each selected to address specific functional requirements of the system. The ESP32 firmware is developed in **Arduino/C++**, responsible for managing sensor data acquisition, wireless communication, and MQTT-based messaging for command reception and data publishing.

The **Data Proxy** is implemented in **Python** using the **FastAPI** framework. It exposes efficient asynchronous endpoints for ingesting sensor data and features a **Tkinter-based GUI** for real-time monitoring and control (it can send MQTT message to ESP32). The **Analytics Backend**, also developed in Python, utilizes the **Flask** framework to serve dashboards and RESTful APIs. This backend incorporates **NumPy** and **Pandas** for data manipulation, and **ARIMA** for predictive modeling. **Threading mechanisms** enable concurrent execution of computational tasks, thereby enhancing responsiveness and system throughput.

For client-side visualization, the system employs **JavaScript** with the **Chart.js** library, embedded within responsive **HTML/CSS** templates. Data visualizations are updated in real-time through periodic **HTTP requests** to the exposed Flask API. A dedicated **Telegram bot**, developed using the **python-telegram-bot** library, enables mobile interaction with the system via Telegram's API. The Telegram bot is used to manually set the status of HVAC. If the HVAC status is on the alarm are generated, else they are not generated. It supports asynchronous message handling and offers an **inline keyboard interface** for intuitive control. Time-series database operations are facilitated using **InfluxDB Python client libraries**, with advanced querying performed using the **Flux** language.

C. Data Processing Pipeline

The system's data pipeline transforms raw sensor inputs into actionable insights through a structured sequence of operations. Timestamps are assigned at the point of acquisition on the ESP32 to ensure accurate latency tracking. Incoming data undergoes validation and cleaning procedures to filter out anomalous values that could indicate sensor malfunction. Following standardization into 30-second aggregation windows, validated data is stored in measurement-specific **InfluxDB** buckets.

Flux queries are utilized for temporal aggregation and pivot operations, preparing the data for downstream analytics. The pipeline calculates statistical correlations between indoor and outdoor temperatures to infer potential inefficiencies in energy usage.

A central component of the pipeline is its **predictive analytics** capability. Historical temperature data is used to fit ARIMA model that forecast indoor temperatures at future time horizons of 1, 10, 20, and 30 minutes. These forecasts enable anticipatory HVAC control strategies. Model performance is continuously evaluated using metrics such as **Mean Absolute Error (MAE)**, **Mean Squared Error (MSE)**, and **Root Mean Squared Error (RMSE)**, which are exposed through a dedicated dashboard. In addition, the system detects potential

energy waste by analyzing indoor-outdoor temperature differentials and can trigger LED-based alerts via MQTT when significant inefficiencies are identified.

D. Communication Protocols

The system employs a hybrid communication model, using protocols suited to the specific needs of data flow and control. **MQTT** is used exclusively for transmitting control commands and receiving real-time status updates. The MQTT broker operates on a local server and enforces authentication through credentials configured in the ESP32 firmware. Key MQTT topics include `hvac/control` (system activation), `hvac/led` (indicator control), and `hvac/sampling_rate` (sensor frequency adjustment).

HTTP/REST complements MQTT by facilitating structured data exchange. The Data Proxy provides RESTful endpoints for data ingestion, while the Analytics Backend exposes endpoints for historical data retrieval. All API communications are conducted using standardized **JSON** formats. For database access, the system leverages both the **InfluxDB REST API** and specialized client libraries, with data insertion and query operations conducted using the **Write API** and **Flux query language**, respectively.

E. Prediction of temperatures

The core of the system's predictive analytics capability is realized through a machine learning-based temperature forecasting module, which leverages an ARIMA (AutoRegressive Integrated Moving Average) model for time series prediction. This module is designed to anticipate future indoor temperature trends by analyzing historical sensor data, thereby enabling proactive detection of potential energy waste scenarios in HVAC operation.

The prediction process begins with the acquisition of time-indexed temperature data, comprising both indoor and outdoor measurements. The ARIMA model is configured to use the indoor temperature as the primary target variable, while the outdoor temperature serves as an exogenous input, capturing the influence of external environmental conditions on indoor climate dynamics. The model selection and parameterization are automated using the `auto_arima` algorithm, which systematically identifies the optimal ARIMA configuration based on the observed data.

Prior to model fitting, the input data undergoes rigorous validation and cleaning to remove missing or anomalous values, ensuring the robustness of the forecasting process. The model is then trained on the historical data, and multi-step ahead forecasts are generated for user-defined horizons (e.g., 1, 10, 20, and 30 minutes). For each prediction interval, the system assumes that the outdoor temperature remains constant at its most recently observed value, reflecting a short-term forecasting context.

A key feature of the prediction module is its integration with the system's energy waste detection logic. The predicted indoor temperatures are compared against the current measured value, and if the deviation exceeds a configurable threshold while the HVAC system is active, an energy waste

alert is triggered. This mechanism ensures that alerts are only generated in operationally relevant scenarios, thereby minimizing false positives and enhancing the practical utility of the system.

The prediction results, including historical and forecasted temperature values, associated timestamps, and alert status, are systematically recorded and made available for visualization and further analysis. This approach not only supports real-time monitoring and control but also facilitates retrospective evaluation of model performance using standard metrics such as Mean Absolute Error (MAE), Mean Squared Error (MSE), and Root Mean Squared Error (RMSE).

Overall, the ARIMA-based predictor constitutes a critical component of the system, enabling data-driven, anticipatory control strategies that enhance energy efficiency and operational reliability in HVAC management.

F. User Interface Design

The user interface is designed to support both local operators and remote users. The **primary web dashboard** provides real-time temperature monitoring, forecast visualizations, and system status indicators. Forecasts for the next 30 minutes are displayed alongside current sensor readings, with comparative data from external weather APIs used for sensor validation. Color-coded indicators (e.g., green for active, gray for inactive) offer intuitive understanding of HVAC status (Figure 5).

For local users, a **Tkinter GUI** panel provides direct access to monitoring and control functions, including connection status indicators and predefined control options (Figure 4).

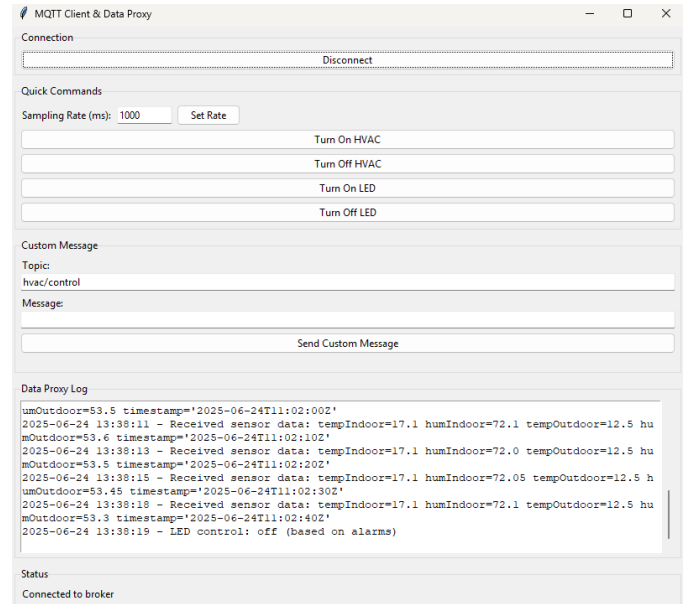


Fig. 4: Data proxy Tkinter interface

For remote interaction, a **Telegram bot** allows mobile users to control and monitor the system without additional applications. The bot features a command-based interface with inline buttons, supports real-time status updates, and prevents unauthorized access through chat ID validation (Figure 6).

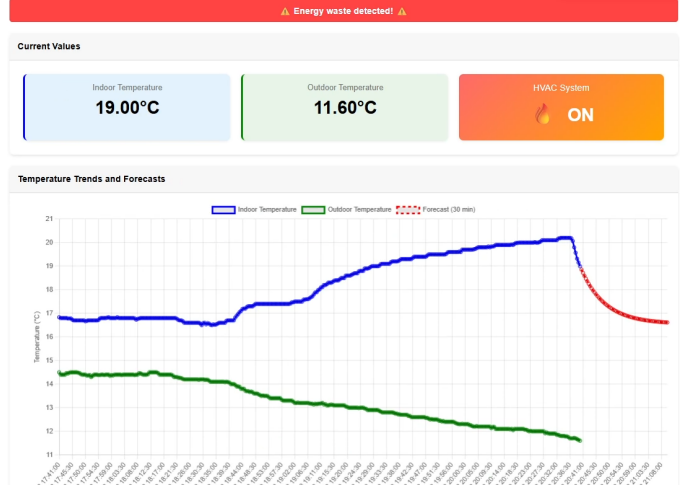
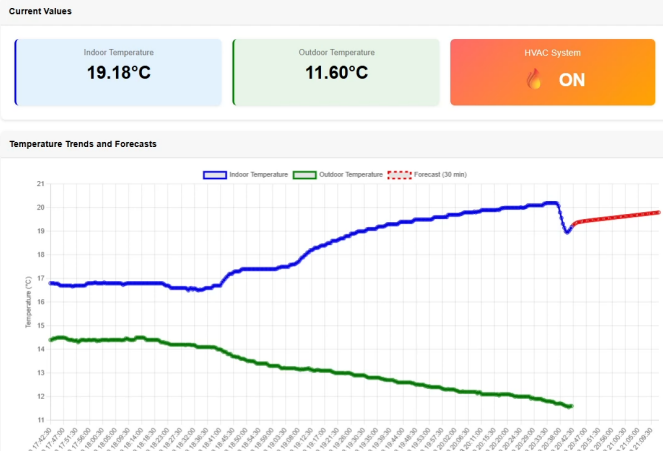


Fig. 5: Dashboard with and without energy waste detected

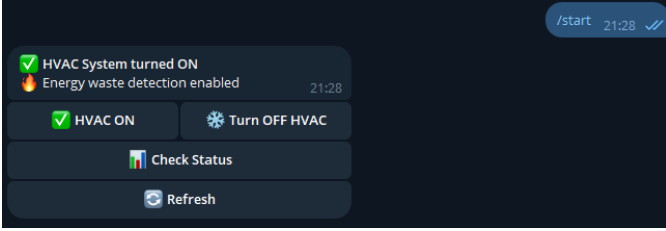


Fig. 6: Telegram bot

Finally, the system supports **Grafana** integration for advanced, customizable dashboards. These dashboards enable historical trend analysis and incorporate alerting mechanisms for waste detection.

IV. RESULTS

This section presents the experimental results obtained from the IoT-based HVAC monitoring system. The following figures illustrate the system's behavior during a representative 10-hour observation period, with data visualized using Grafana.

Figure 7 displays the temperature readings collected from both the indoor and outdoor sensors over time. This plot provides an initial overview of the thermal dynamics captured by the system.

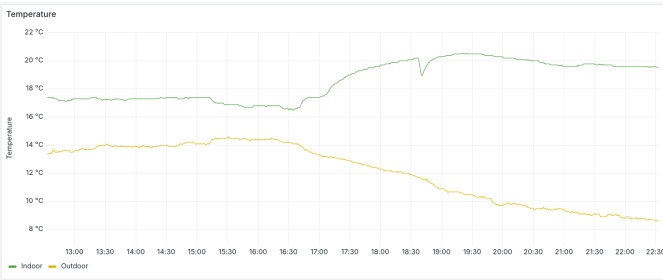


Fig. 7: Indoor and outdoor temperature trends

Figure 8 shows the cumulative number of HVAC waste alerts triggered per minute. This graph highlights the frequency

and distribution of detected inefficiencies throughout the monitoring period.

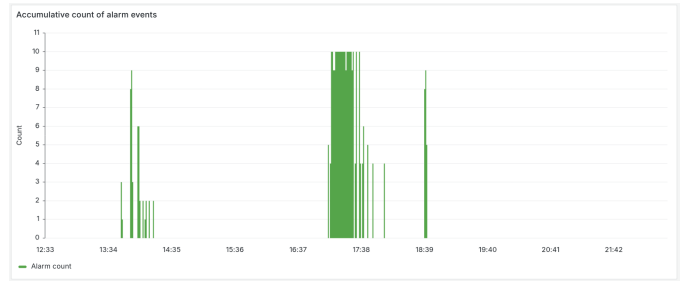


Fig. 8: Cumulative count of HVAC waste alerts per minute

In Figure 9, indoor and outdoor temperature trends are compared, with red highlights indicating instances where an HVAC waste condition was detected. This visualization enables direct correlation between temperature differentials and alert generation.

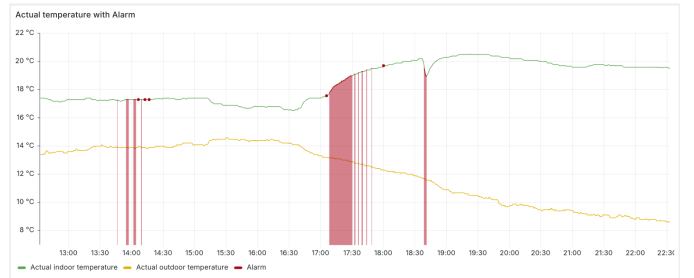


Fig. 9: Temperature comparison with HVAC waste alert highlights

The system generates temperature forecasts every 10 seconds. Each prediction is timestamped and stored in the database, with forecast horizons of 1, 10, 20, and 30 minutes. Figure 10 illustrates the predictions generated at each of these horizons, plotted alongside the actual measured temperature values. For example, a prediction made at 14:00 for the 30-

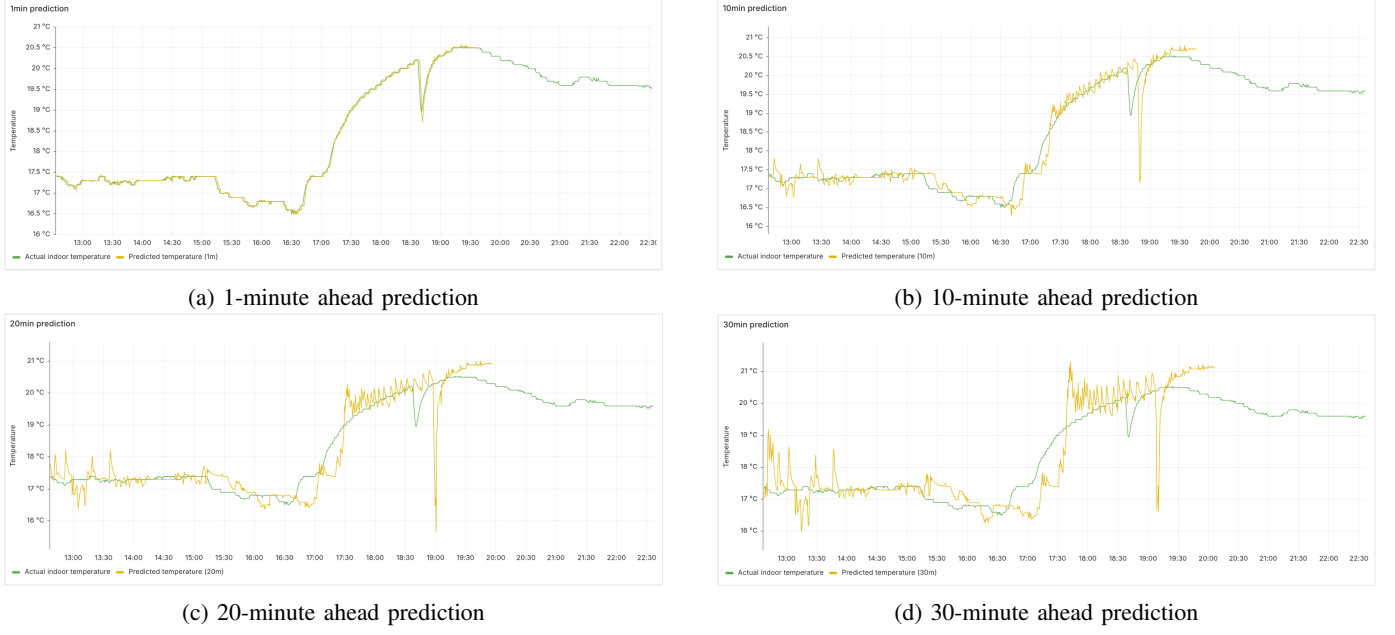


Fig. 10: Temperature forecasts at various time horizons



Fig. 11: Forecasting performance at increasing prediction intervals

minute horizon is plotted against the real temperature recorded at 14:30, allowing visual evaluation of prediction accuracy.

For each stored forecast, performance metrics were computed using Mean Absolute Error (MAE), Mean Squared Error (MSE), and Root Mean Squared Error (RMSE). Figure 11 presents the model's predictive performance across the four forecast intervals. As expected, shorter prediction horizons exhibit lower error values, while predictive accuracy deteriorates as the horizon extends.

Finally, Figure 12 presents the measured data transmission latency for each sensor reading. Latency is computed as the elapsed time between the message timestamp generated by the ESP32 and the final write operation to the InfluxDB database via the proxy. The average end-to-end latency is approximately 500 milliseconds, indicating near real-time processing capability.

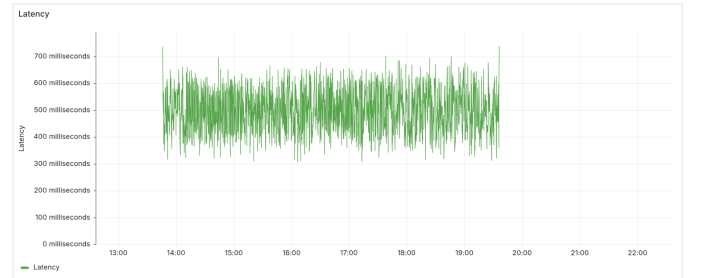


Fig. 12: Data transmission latency over time

A. Discussion of Results

The temperature profiles depicted in Figure 7 reveal a clear correlation between indoor and outdoor conditions, with indoor temperatures exhibiting dampened fluctuations compared to outdoor ones. This behavior suggests effective thermal buffering by the indoor environment. The cumulative waste alerts shown in Figure 8 occur primarily during significant temperature differentials, which is further confirmed by the

direct overlay of alerts in Figure 9. Thus, the system accurately identifies periods of potential energy inefficiency.

The predictive models demonstrate commendable short-term forecasting accuracy, as evinced by the low error values (MAE, MSE, and RMSE) for 1-minute ahead predictions in Figure 11. However, as forecast horizons extend to 30 minutes, the performance metrics deteriorate, reflecting increasing uncertainty intrinsic to longer-term predictions. Nonetheless, the system maintains an acceptable error margin for operational use within the first 10 minutes, enabling proactive HVAC control.

The observed average latency of approximately 500 ms (Figure 12) confirms that the end-to-end communication, spanning sensor reading, network transmission, proxy handling, and database storage—operates within the real-time constraints required for effective control action. This latency level supports timely forecast-driven adjustments and responsive alerting mechanisms.

Overall, the results validate the system’s capability to continuously monitor environmental conditions, generate reliable short-term forecasts, detect instances of energy waste, and maintain responsiveness compatible with real-time operational requirements.

REFERENCES

- [1] M. González-Torres, L. Pérez-Lombard, Juan F. Coronel, Ismael R. Maestre, and Da Yan. A review on buildings energy information: Trends, end-uses, fuels and drivers. *Energy Reports*, 8:626–637, 2022.